CSC 255

Unit 7 Programming Assignment 1 [50 points]

Huffman Coding

Read about Huffman Coding in one of the course textbooks.

Watch this video by Reducible.

Write a program to implement file compression using Huffman's Algorithm (see algorithm below). The program must use the heap (binary tree) implementation of a priority queue provided by your textbook's source code.

Use your program to compress the provided input file (*huffman.in*). **[10 points]**

Your program should write the results to an output file (*huffman.out*). **[5 points]**

Your program will then decompress (see algorithm below) the output file (*huffman.out*) and confirm the results match the original input file (*huffman.in*). **[10 points]**

Implement Huffman's Algorithm (pseudocode provided below) by manually tracing through the steps used to build a Huffman coding tree (see *Huffman Algorithm Trace* document) on the following passage: **`CHATBOT CHEAT`** **[10 points]**

Use your program to confirm the results of the coding tree. Additionally, the program should generate a table labeling each symbol with its associated encoding, frequency, and total bits, and give the total frequency and bits sum as output to the console or separate output file (*table.out*). See an example of a similar table below (**Table 1**). **[10 points]**

Compute the total number of bits of the encoded passage. **[1 point]**

How many bits are required if it is not encoded? Each symbol requires a 3 bit code since there are $2^3 = 8$ distinct symbols. **[1 point]**

The **average length (*L*)** of an encoded symbol (before or after compression) can be calculated by taking the following sum, where $P(x_i)$ is the probability of symbol x_i and n is number of symbols in the passage:

$$L = \sum_{i=1}^{n} P(x_i) \times \text{length}(x_i)$$

We use $P(x_i) = \frac{freq(x_i)}{n}$ for the calculation above.
If every symbol of the passage requires 4 bits (or 8 in ASCII) to encode then $L = 4$ bits per symbol. The total number of bits to encode the passage is $T = L \times n$.
The **lower bound or entropy (*H*)** of compression (in bits per symbol) is based on the following formula:

$$H = \sum_{i=1}^{n} -P(x_i) \times \log\left[P(x_i)\right]$$

The theoretical lower bound on the total number of bits to encode using compression is $T_H = H \times n$, and the actual lower bound on the total number of bits to encode using compression is $\lceil T_H \rceil$.
The **redundancy (*R*)** of the encoding is defined as the difference between average length (after compression) and entropy:

$$R = L - H$$

Redundancy measures the inefficiency of encoding. A highly efficient encoding would yield a redundancy of 0.
What is the redundancy (in bits) of the passage? Find this by computing $T - \lceil T_H \rceil$. **[3 points]**

Huffman's Algorithm

```
Input: passage string X of length n

Output: binary encoding tree for X

function huffman(X):

      Compute the frequency f(c) of each character c of X.

      Initialize a priority queue Q.

      for each character c in X do

            Create a single-node binary tree T storing c.

            Insert T into Q with key f(c).
      while length(Q) > 1 do

            (f1,T1)= Q.remove min()

            (f2,T2)= Q.remove min()

            Create a new binary tree T with left subtree T1 and right subtree T2.

            Insert T into Q with key f1 + f2.
      (f,T)= Q.remove min()

      return tree T
```

Decompression Algorithm

```
Input: binary encoded string E of length n

Output: original passage string X

function decode(E):

      Initialize an empty string S.

      Initialize an iterator P to the root of the binary encoding tree.

      Initialize n to the length of E.
```

```
for i = 0 to n do
      if E[i] is '0' then
          P = P.left
      else
          P = P.right
      if P.left and P.right are Empty then
          S += P.symbol
          Reinitialize P to the root of the binary encoding tree.
return S
```

Character	Code	Frequency	Total Bits
a	000	10	30
e	001	15	45
i	010	12	36
s	011	3	9
t	100	4	12
space	101	13	39
newline	110	1	3
Total			174

Table 1